

CSE488: Detailed Notes on Apache SPARK

1. Limitations of MapReduce and the Emergence of Spark

Traditional MapReduce programming has several limitations:

- Verbose and low-level APIs make it hard to develop complex workflows.
- Inefficiency due to disk I/O after every map and the reduce stage.
- Poor support for iterative and interactive computations.

Apache Spark was introduced to address these issues. It provides:

- In-memory data processing (reducing I/O overhead).
- High-level APIs in Scala, Java, and Python.
- Flexible DAG execution engine that supports general data-flow applications.

2. Spark Architecture Overview

- **Driver Program:** Orchestrates execution, creates RDDs, transformations, and actions.
- **Cluster Manager:** Allocates resources (YARN, Mesos, or standalone).
- **Executors:** Run tasks and cache data.
- **DAG Scheduler & Task Scheduler:** Convert logical plans to physical execution plans.

3. Core Concept: RDD (Resilient Distributed Dataset)

- **Immutable**, distributed collection of objects.
- Supports **lazy transformations** and **in-memory caching**.
- Can be created from Hadoop, local collections, or transformations of existing RDDs.

Key Properties:

- **Fault-tolerant:** Automatically rebuilt upon failure
- **Partitioned:** a distributed collection of objects
- **Lazy evaluation:** Only computed when an action is triggered
- Supports transformations and actions

4. RDD Transformations

Transformations are lazy operations that define a new RDD based on the current one.

Common Transformations with Examples:

- `map(func)`: Apply a function to each element

```
rdd.map(lambda x: x * 2)
```

- `filter(func)`: Keep elements where function returns true

```
rdd.filter(lambda x: x % 2 == 0)
```

- `flatMap(func)`: Similar to map, but flattens the result

```
rdd.flatMap(lambda line: line.split())
```

- `union(otherRDD)`: Combines two RDDs
- `distinct()`: Removes duplicates
- `intersection(otherRDD)`: Elements common in both
- `subtract(otherRDD)`: Elements in one but not in other
- `groupByKey(func)`: Groups by a function

5. Key-Based Transformations

- `groupByKey()`: Groups values for each key (inefficient due to shuffle)

```
rdd.groupByKey().mapValues(list)
```

- `reduceByKey(func)`: Reduces values for each key using the given function (efficient)

```
rdd.reduceByKey(lambda x, y: x + y)
```

- `combineByKey(createCombiner, mergeValue, mergeCombiners)`: Advanced aggregation with type flexibility

Feature	<code>groupByKey()</code>	<code>reduceByKey(func)</code>
Operation	Groups values into a list	Aggregates values
Shuffling	High (all values shuffled)	Less (partial reduce before shuffle)

Feature	groupByKey()	reduceByKey(func)
Performance	Slower on large data	Faster and more scalable
Use Case	When you need all values as a list	When you can reduce/combine

Example: Average by key

```
createCombiner = lambda v: (v, 1)
```

```
mergeValue = lambda acc, v: (acc[0] + v, acc[1] + 1)
```

```
mergeCombiners = lambda acc1, acc2: (acc1[0] + acc2[0], acc1[1] + acc2[1])
```

```
rdd.combineByKey(createCombiner, mergeValue, mergeCombiners)
```

6. Spark Actions

Actions trigger computation and return results to the driver or write to storage.

Common RDD Actions:

- `collect()`: Return all elements
- `count()`: Number of elements
- `first()`: First element
- `take(n)`: First n elements
- `reduce(func)`: Aggregate using a function
- `foreach(func)`: Perform action on each element
- `saveAsTextFile(path)`: Write to text file

DataFrame Actions:

- `show(n)`, `collect()`, `count()`, `first()`, `toPandas()`, `write` (e.g., CSV, Parquet)

7. Narrow vs Wide Transformations

A **narrow transformation** is one where each output partition depends on only one input partition. There is no data movement (shuffle) between partitions across nodes.

A **wide transformation** is one where the output partitions depend on multiple input partitions. This typically involves shuffling data across the network.

Feature	Narrow Transformation	Wide Transformation
Data Movement	No	Yes (shuffle across nodes)
Partition Dependency	Single parent partition	Multiple parent partitions
Performance	Faster	Slower (due to shuffle)
Execution	Can be pipelined	Requires stage boundaries
Examples	map, filter, flatMap	reduceByKey, join, groupByKey

Narrow Example:

```
rdd.map(lambda x: x * 2)
```

Wide Example:

```
rdd.reduceByKey(lambda x, y: x + y)
```

8. Higher-Level APIs: DataFrame and Dataset

- **DataFrame**: Distributed collection of data with named columns (like a table).
- **Dataset** (Scala/Java only): Typed version of DataFrame with compile-time type safety.

Feature	RDD	DataFrame	Dataset
Type Safety	Yes	No	Yes (Scala/Java only)
Performance	Low	High	High
API Support	All	All	Scala/Java only

9. Spark Ecosystem Components

- **Spark SQL:** For structured data and SQL queries
- **Spark Streaming:** Real-time data stream processing
- **MLlib:** Machine learning library (classification, regression, clustering)
- **GraphX:** Graph processing engine with Pregel-like API

10. Further Reading and Tools

- Spark Documentation: <https://spark.apache.org/docs/latest/>
- Book: *Learning Spark* by Jules S. Damji et al.
- Tools: Spark UI, Jupyter Notebooks, Databricks